LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 47. Short Circuit

Section 1: Lecture Summary

This lecture focuses on the concept of **short circuit evaluation** in Python, especially in relation to logical operators **and** and **or**. It starts with a quick review of logical operators and compound conditional statements. Then it delves into how short circuiting works with these operators, explaining that evaluation can stop as soon as the outcome is known without checking all conditions. The lecture also introduces **non-boolean conditions** where logical operators can return non-boolean values based on the truthiness of operands, illustrating this behavior through examples and demonstrations in Jupyter Notebook. The concept is explained in detail and highlighted as important for both programming practice and interview preparation.

Section 2: Key Concepts and Explanations

1. Logical Operators Revisited

- Python has three basic logical operators: **and**, **or**, and **not**.

- These operators typically return boolean values: `True` or `False`.

- They are used in compound conditions combined in `if` statements for decision making.

2. Short Circuit Evaluation

- Refers to the mechanism where evaluation of conditions stops as soon as the overall result is determined.

Short Circuit with `and` operator:

- Logical `and` returns `True` only if both conditions are true.

- If the first condition is **False**, Python does **not** evaluate the second condition because the result will always be false (short circuit).

- If the first condition is **True**, it evaluates the second to determine the final result.

- Summary:

  - If `cond1` is False → result is False immediately, no `cond2` check.

  - If `cond1` is True → evaluates `cond2`; result depends on `cond2`.

Short Circuit with `or` operator:

- Logical `or` returns `True` if any condition is true.

- If the first condition is **True**, Python does **not** evaluate the second condition as the overall result is already True.

- If the first condition is **False**, it evaluates the second condition to find result.

- Summary:

  - If `cond1` is True → result is True immediately, no `cond2` check.

  - If `cond1` is False → evaluates `cond2`; result depends on `cond2`.

3. Truth Tables for Understanding

- For `and`:

cond1	cond2	cond1 and cond2
True	True	True
True	False	False
False	True	False (no check of cond2)
False	False	False (no check of cond2)

- For `or`:

cond1	cond2	cond1 or cond2
True	True	True (no check of cond2)

True	False	True (no check of cond2)
False	True	True
False	False	False

4. Non-Boolean Conditions in Logical Operations

- Python logical operators can be used with non-boolean operands.

- In these cases, Python returns one of the operands, not just `True` or `False`.

- For `and` operator:

  - Returns the first operand if it is falsy (e.g., 0, `False`).

  - Otherwise, returns the second operand.

- For `or` operator:

  - Returns the first operand if it is truthy.

  - Otherwise, returns the second operand.

- Truthiness concept:

  - `False` means: `0`, `None`, `''`, empty containers, etc.

  - `True` means: any value not considered falsy (any number other than zero, non-empty containers, etc.)

Section 3: Example Code and Use Cases

Variables

Short circuit with 'and' - first condition false

Short circuit with 'and' - first condition true, second false

Short circuit with 'and' - both conditions true

Short circuit with 'or' - first condition true, second not evaluated

Short circuit with 'or' - first condition false, second true

Non-boolean conditions examples

```
A = 10
B = 5
C = 3

result = (A < B) and (A > C)  # First is False, second not evaluated
print(result)  # Output: False

result = (A > B) and (A < C)  # First True, second False, both evaluated
print(result)  # Output: False

result = (A > B) and (A > C)  # Both True
print(result)  # Output: True

result = (A > B) or (A > C)
print(result)  # Output: True

result = (A < B) or (A > C)
print(result)  # Output: True

print(5 and 10)   # Output: 10 (both truthy, 'and' returns second)
print(5 and 0)    # Output: 0  (first truthy, second falsy, returns 0)
print(0 and 10)   # Output: 0  (first falsy, returns 0 without checking
second)

print(5 or 10)    # Output: 5  (first truthy, returns 5 without checking
second)
print(0 or 10)    # Output: 10 (first falsy, returns second operand)
```

Explanation:

- The `and` operator returns the first falsy operand encountered or the last operand if none are falsy.

- The `or` operator returns the first truthy operand or the last operand if all are falsy.

- These behaviors illustrate short circuit evaluation and boolean context in Python.

Section 4: Key Takeaways

- **Short circuiting optimizes evaluation** by skipping unnecessary condition checks.

- For `and`: evaluation stops if the first condition is False.

- For `or`: evaluation stops if the first condition is True.

- Logical operators can be used with non-boolean values, returning one of the operands based on their truthiness.

- Understanding short circuit can prevent bugs and improve efficiency in conditional statements.

- Short circuit behavior is common to many programming languages, not just Python.

- The concept is frequently asked in technical interviews and certification exams.

- Practical testing via tools like Jupyter Notebook reinforces understanding.

These notes provide a comprehensive overview for mastering short circuit evaluation with emphasis on real code examples and clear explanation of underlying logic.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.

Short Circuit

Short Circuit

Short Circuit

Revisit Logical Operators

Short Circuit

Revisit Logical Operators

Short Circuit ?

Short Circuit

Revisit Logical Operators

Short Circuit ?

‘and’

Short Circuit

- Revisit Logical Operators
- Short Circuit ?
  - 'and'
  - 'or'

Short Circuit

- Revisit Logical Operators
- Short Circuit ?
  - 'and'
  - 'or'
- Non-Boolean Condition

Short Circuit

Revisit Logical Operators

Logical Operators

and	AND
or	OR
not	NOT

Logical Operators

and	AND
or	OR
not	NOT

Boolean

Logical Operators

and	AND
or	OR
not	NOT

True/False

Logical Operators

and
or
not

Conditional Statement

if <condition> :

else :

and
or
not

Conditional Statement

if <cond1> <cond2> :

else :

and
or
not

Conditional Statement

if <cond1> and <cond2> :
______
______
else :
______
______

and
or
not

Conditional Statement

if <cond1> or <cond2> :
______
______
else :
______
______

and
or
not

Short Circuit with “and”

and
or
not

Short Circuit ‘and’

and
or
not

Short Circuit 'and'

if <cond1> and <cond2> :

and
or
not

Short Circuit 'and'

a = 10 b = 5 c = 3

if <cond1> and <cond2> :

and
or
not

Short Circuit 'and'

a = 10 b = 5 c = 3

if a > b and a > c :

- and
- or
- not

Short Circuit 'and'

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

a = 10 b = 5 c = 3

if a > b and a > c :

- and
- or
- not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10   b = 5   c = 3

```
if  a > b  and  a > c :
```

T

- and
- or
- not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10 b = 5 c = 3

if a > b and a > c :

T T

and
or
not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10 b = 5 c = 3

T ← if a > b and a > c :

T T

and
or
not

Short Circuit 'and'

a = 10   b = 5   c = 3

if   and   :

and
or
not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10   b = 5   c = 3

if a < b and a > c :

F

and
or
not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10   b = 5   c = 3

F ← if a < b and a > c :
F

- and
- or
- not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10   b = 5   c = 3

F ← if a < b and $\cancel{a > c}$:
F

- and
- or
- not

and

cond1	cond2	cond1 and cond2
T	T	T
T	F	F
F	T	F
F	F	F

Truth Table

Short Circuit 'and'

a = 10   b = 5   c = 3

F ← if a > b and a < c :

T (a > b)   F (a < c)

and
or
not

Short Circuit 'and'

Short Circuit 'and'

cond1 and cond2 :

Short Circuit 'and'

cond1 and cond2 :

T

Short Circuit 'and'

cond1 and cond2 :
T T

Short Circuit 'and'

T ← cond1 and cond2 :

T T

Short Circuit 'and'

T ← cond1 and cond2 :

T T

Short Circuit ‘and’

T ← cond1 and cond2 :
T T

cond1 and cond2 :

Short Circuit ‘and’

T ← cond1 and cond2 :
T T

cond1 and cond2 :
F

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ×

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ×

cond1 and cond2 :

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ✕

cond1 and cond2 :
T

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ✕

cond1 and cond2 :
T F

Short Circuit ‘and’

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ✕

F ← cond1 and cond2 :
T F

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ×

F ← cond1 and cond2 :
T F

Short Circuit 'and'

T ← cond1 and cond2 :
T T

F ← cond1 and cond2 :
F ×

F ← cond1 and cond2 :
T F

Short Circuit

- and
- or
- not

Short Circuit

Short Circuit with “or”

- and
- or
- not

Short Circuit 'or'

if <cond1> or <cond2> :

and
or
not

Short Circuit 'or'

a = 10 b = 5 c = 3

if <cond1> or <cond2> :

and
or
not

Short Circuit 'or'

a = 10 b = 5 c = 3

```
if a > b or a > c:
```

and
or
not

Short Circuit 'or'

a = 10 b = 5 c = 3

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

```
if a > b or a > c:
```

and
or
not

Short Circuit 'or'

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

a = 10 b = 5 c = 3

if a > b or a > c :
T

and
or
not

Short Circuit 'or'

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

a = 10 b = 5 c = 3

T ← if a > b or a > c :
T

and
or
not

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

Short Circuit 'or'

a = 10   b = 5   c = 3

T ← if a > b or a > c :

a > b: T

and
or
not

Short Circuit 'or'

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

a = 10 b = 5 c = 3

if or :

- and
- or
- not

Short Circuit 'or'

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

a = 10 b = 5 c = 3

if a < b or a > c :

F T

- and
- or
- not

or

cond1	cond2	cond1 or cond2
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table

Short Circuit 'or'

a = 10 b = 5 c = 3

T ← if a < b or a > c :

F T

and

or

not

Short Circuit 'or'

Short Circuit 'or'

cond1 or cond2 :

Short Circuit 'or'

cond1 or cond2 :
T

Short Circuit 'or'

T ← **cond1** **or** **cond2 :**

T

Short Circuit 'or'

T ← **cond1** **or** **cond2 :**

T ✗

Short Circuit 'or'

T ← cond1 or cond2 :
T ×

Short Circuit 'or'

T ← cond1 or cond2 :
T ×

cond1 or cond2 :

Short Circuit ‘or’

T ← cond1 or cond2 :

T ✕

cond1 or cond2 :

F

Short Circuit 'or'

T ← cond1 or cond2 :
T ×

cond1 or cond2 :
F T

Short Circuit 'or'

T ← cond1 or cond2 :
T ×

T ← cond1 or cond2 :
F T

Short Circuit 'or'

T ← cond1 or cond2 :
T ✕

T ← cond1 or cond2 :
F T

Short Circuit 'or'

T ← cond1 or cond2 :
T ✕

T ← cond1 or cond2 :
F T

Short Circuit

Short Circuit

Non-Boolean Condition

Short Circuit
Non-Boolean Condition

Boolean

Short Circuit
Non-Boolean Condition

Boolean

False → 0

Short Circuit
Non-Boolean Condition

Boolean

False → 0

True → 1

Short Circuit
Non-Boolean Condition

Boolean

False → 0

True → any other value

Short Circuit
Non-Boolean Condition

Boolean

False → **0**

True → **any other value**

[1 , -1 , 10, -15, 7]

Short Circuit

Non-Boolean Condition

Short Circuit

Non-Boolean Condition

```
a > b and a > c
```

Short Circuit

Non-Boolean Condition

5 and a > c

Short Circuit

Non-Boolean Condition

5 and 10
T

Short Circuit

Non-Boolean Condition

5 and 10

T T

Short Circuit

Non-Boolean Condition

10 ← 5 and 10

T T

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

5 and 0

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

5 and 0
T

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

5 and 0
T F

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

0 ← 5 and 0
T F

Short Circuit

Non-Boolean Condition

10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
		0	and	10

Short Circuit
Non-Boolean Condition

10 ← 5 and 10
T T

0 ← 5 and 0
T F

0 ← 0 and 10
F

Short Circuit
Non-Boolean Condition

10 ← 5 and 10
T T

0 ← 5 and 0
T F

0 ← 0 and 10
F

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

0 ← 5 and 0
T F

0 ← 0 and 10
F ✕

Short Circuit

Non-Boolean Condition

10 ← 5 and 10
T T

0 ← 5 and 0
T F

0 ← 0 and 10
F ✕

5 or 10

Short Circuit

Non-Boolean Condition

10 ←	5 (T)	and	10 (T)
0 ←	5 (T)	and	0 (F)
0 ←	0 (F)	and	10 (✕)
	5 (T)	or	10

Short Circuit

Non-Boolean Condition

10 ←	5 (T)	and	10 (T)
0 ←	5 (T)	and	0 (F)
0 ←	0 (F)	and	10 (✕)
5 ←	5 (T)	or	10

Short Circuit
Non-Boolean Condition

10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✗)
5	←	5 (T)	or	10 (✗)

Short Circuit
Non-Boolean Condition

10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✗)
5	←	5 (T)	or	10 (✗)
		0	or	10

Short Circuit

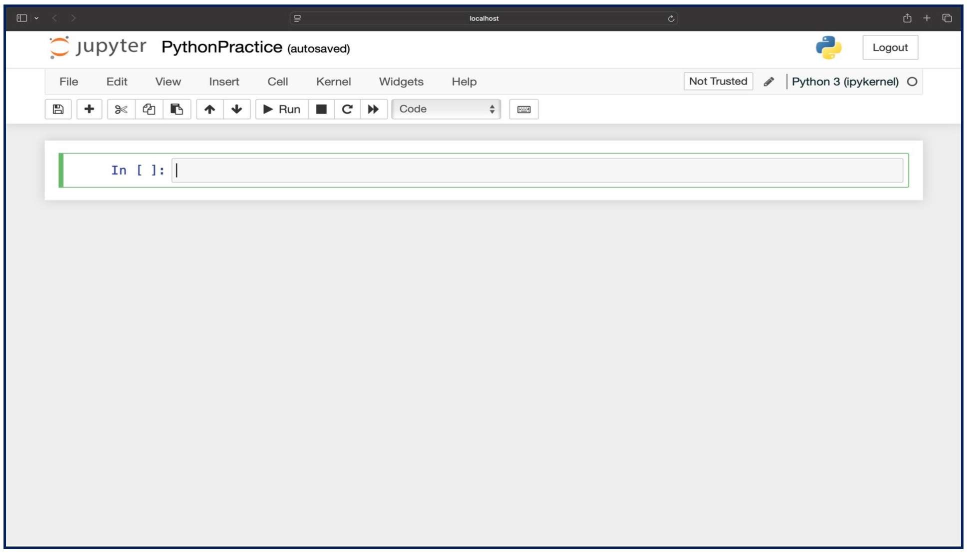
Non-Boolean Condition

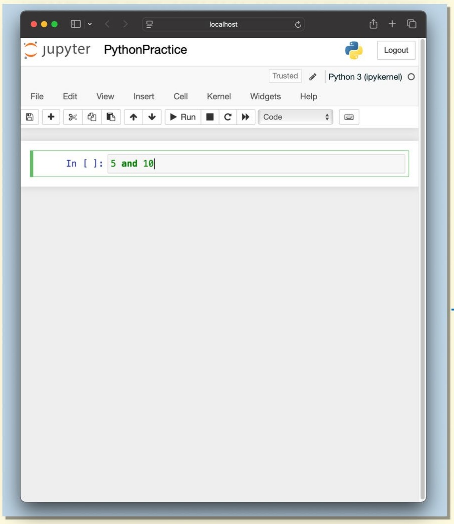
10	←	5	and	10
		T		T
0	←	5	and	0
		T		F
0	←	0	and	10
		F		✕
5	←	5	or	10
		T		✕
		0	or	10
		F		

Short Circuit

Non-Boolean Condition

10	←	5	and	10
		T		T
0	←	5	and	0
		T		F
0	←	0	and	10
		F		✕
5	←	5	or	10
		T		✕
10	←	0	or	10
		F		T

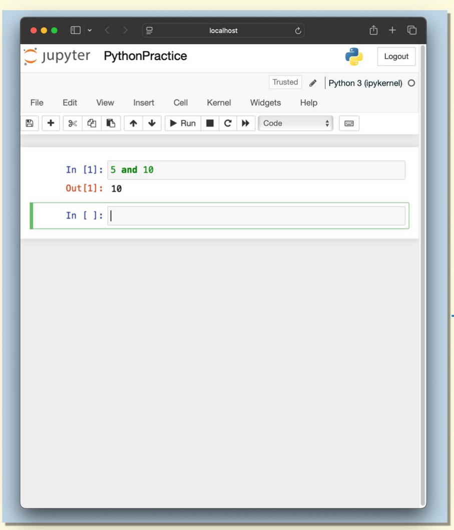

localhost
jupyter
PythonPractice (autosaved)
Logout
File
Edit
View
Insert
Cell
Kernel
Widgets
Help
Not Trusted
Python 3 (ipykernel)
Run
Code
In [ ]:



Short Circuit

Non-Boolean Condition

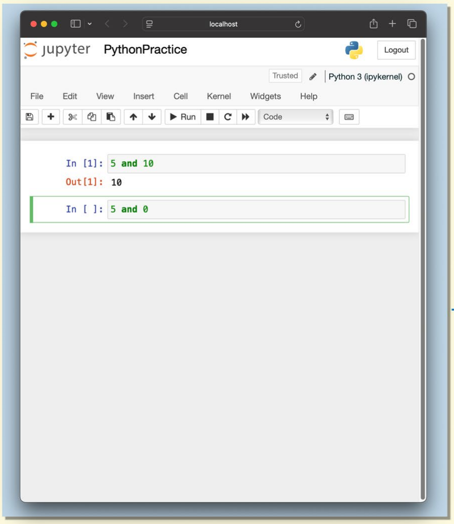
Result		Left	Operator	Right
10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✗)
5	←	5 (T)	or	10 (✗)
10	←	0 (F)	or	10 (T)



Short Circuit

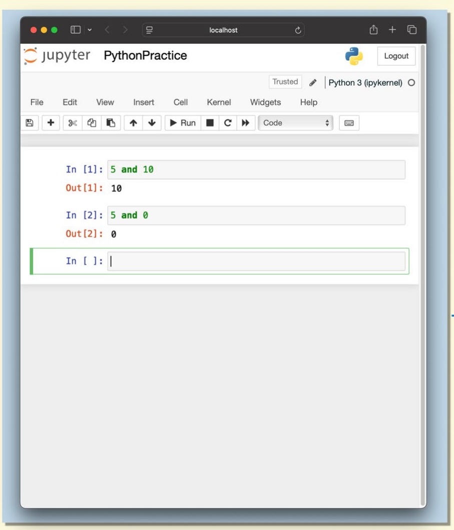
Non-Boolean Condition

Result		Left	Operator	Right
10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✗)
5	←	5 (T)	or	10 (✗)
10	←	0 (F)	or	10 (T)



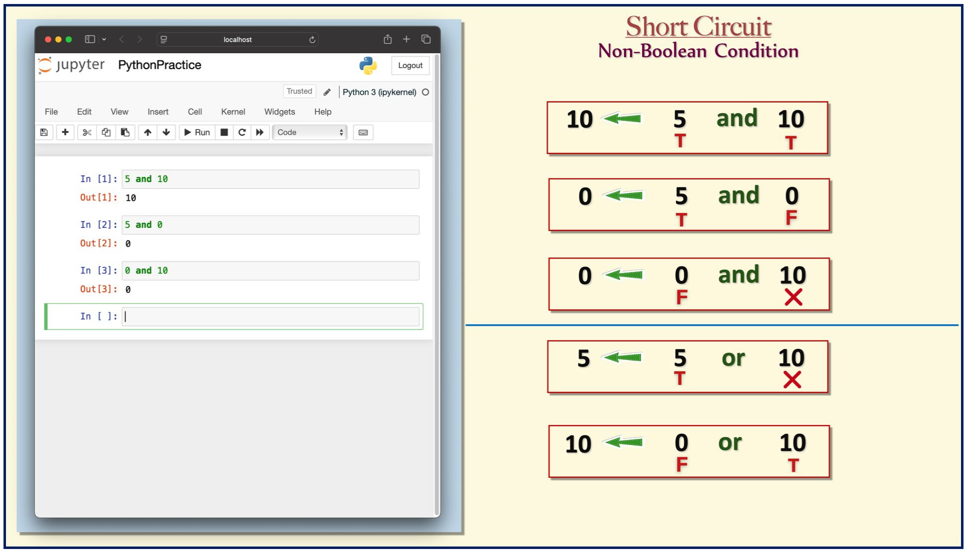
Short Circuit
Non-Boolean Condition

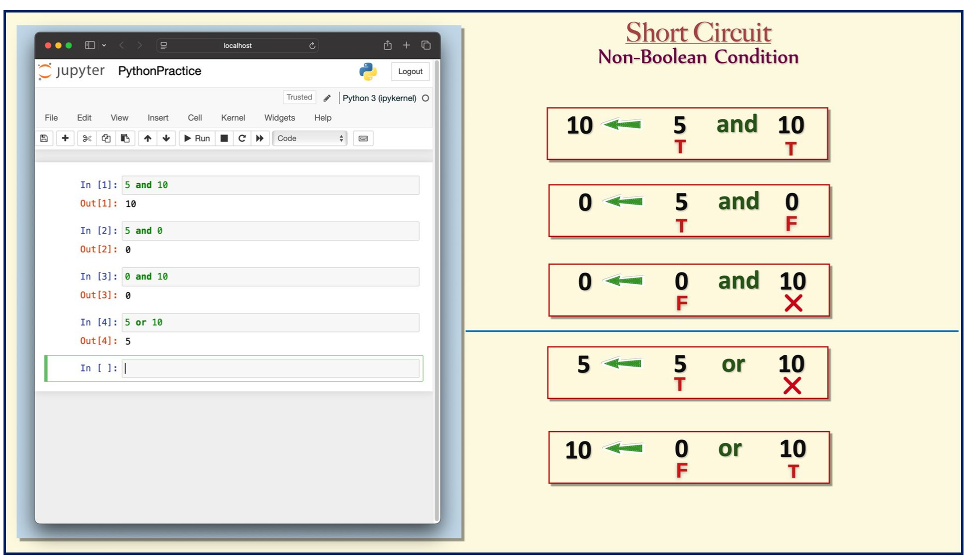
Result		Left	Operator	Right
10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✕)
5	←	5 (T)	or	10 (✕)
10	←	0 (F)	or	10 (T)

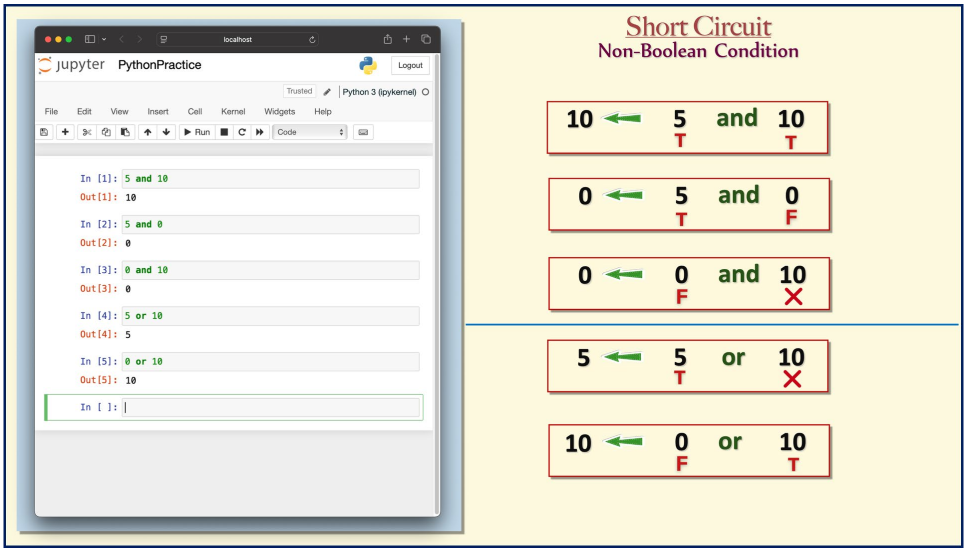


Short Circuit
Non-Boolean Condition

Result		Left	Operator	Right
10	←	5 (T)	and	10 (T)
0	←	5 (T)	and	0 (F)
0	←	0 (F)	and	10 (✕)
5	←	5 (T)	or	10 (✕)
10	←	0 (F)	or	10 (T)


Short Circuit
Non-Boolean Condition
jupyter PythonPractice
In [1]: 5 and 10
Out[1]: 10
In [2]: 5 and 0
Out[2]: 0
In [3]: 0 and 10
Out[3]: 0
10 ← 5 and 10
T T
0 ← 5 and 0
T F
0 ← 0 and 10
F
5 ← 5 or 10
T
10 ← 0 or 10
F T


Short Circuit
Non-Boolean Condition
jupyter PythonPractice
In [1]: 5 and 10
Out[1]: 10
In [2]: 5 and 0
Out[2]: 0
In [3]: 0 and 10
Out[3]: 0
In [4]: 5 or 10
Out[4]: 5
10 ← 5 and 10
T T
0 ← 5 and 0
T F
0 ← 0 and 10
F
5 ← 5 or 10
T
10 ← 0 or 10
F T


localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: 5 and 10
Out[1]: 10
In [2]: 5 and 0
Out[2]: 0
In [3]: 0 and 10
Out[3]: 0
In [4]: 5 or 10
Out[4]: 5
In [5]: 0 or 10
Out[5]: 10
In [ ]:
Short Circuit
Non-Boolean Condition
10 ← 5 and 10
T T
0 ← 5 and 0
T F
0 ← 0 and 10
F
5 ← 5 or 10
T
10 ← 0 or 10
F T